# Desarrollo de Aplicaciones Web

*JavaScript ES6*

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

*Pablo García Tahoces*

# JavaScript ES6

En 2015, ECMAScript hizo una revisión importante del JavaScript, creándose la versión denominada ES6 o ECMAScript 6.

En la actualidad, las versiones actualizadas de los navegadores soportan dicha versión.

| | | | | |
|---|---|---|---|---|
| Chrome 58 | Edge 14 | Firefox 54 | Safari 10 | Opera 55 |
| Jan 2017 | Aug 2016 | Mar 2017 | Jul 2016 | Aug 2018 |

*w3schools*

**Nota→** *Las revisiones han seguido haciéndose cada año, si bien con menor intensidad.*

# ES6. Declaración de variables y constantes

let → Permite declarar variables dentro de un bloque de forma independiente.
No permite re-declaraciones dentro de un mismo bloque.

```
let x= 5

{let x= 10}
```

const → Permite declarar constantes, dentro de un mismo bloque, que no varían su valor a lo largo del programa. No permite reasignaciones.

```
const x=10
```

| | Ámbito | Re-declaración | Reasignación |
|---|---|---|---|
| var | No | Sí | Sí |
| let | Sí | No | Sí |
| const | Sí | No | No |

# ES6. Declaración de variables y constantes

Operador … (spread) → Permite expandir los elementos de un objeto iterable

```
const x = [3,5,1,7,5];

let mV = Math.max(...x);
```

**Nota.-** *Math.max(x)* → *NaN*

# ES6. Lazos “for”.

Lazos for / of → permiten iterar sobre todo tipo de estructuras iterables: arrays, strings, mapas, objetos en general.

```
let cadena= “Mi Cadena”;
let texto= “”;

for (let k of cadena)
   texto += k;                    → texto: MiCadena
```

```
for (let k in cadena)
   texto += k;                    → texto: 0123456789
```

# ES6. Funciones.

Funciones → Se permiten valores por defecto

```
function miFuncion (x, y=5){
   return x+y;
}

miFuncion(3)  → 8
```

Funciones → Se permite un número indefinido de parámetros en la definición, usando el operador "…"

```
function miFuncion(...miPar)
  let k= 0;
  for (let k of miPar)
      s+= k
}

miFuncion(1,2,3,3,3)  → 12
```

# ES6. Funciones “Arrow”.

Funciones “Arrow”→ Funciones pequeñas que no necesitan ni llaves ni *return*.

Función tradicional:

```
function (a){
  return a + 100;
}
```

# ES6. Funciones “Arrow”.

Funciones “Arrow”→ Funciones pequeñas que no necesitan ni llaves ni return.

```
function (a){
  return a + 100;
}
```

Función “Arrow”:

1. Elimina la palabra "function" y coloca la flecha entre el argumento y la llave de apertura.

```
(a) => {
  return a + 100;
}
```

2. Quita las llaves del cuerpo y la palabra "return" → el return está implícito.

```
(a) => a + 100;
```

***Nota.-*** Si tenemos 2 o más líneas se mantendrán tanto las llaves como el return.

3. Suprime los paréntesis de los argumentos

```
a => a + 100;
```

***Nota.-*** *Si tiene varios argumentos o ninguno hay que volver a poner los paréntesis*

Si la función tiene nombre:

```
function miF(a){
  return a + 100;
}
```

⇨

```
miF= a => a + 100;
```

*Pablo García Tahoces*

# ES6. Funciones “Arrow”.

Funciones “Arrow”→ Funciones pequeñas que no necesitan ni llaves ni return.

```
<h2>Funciones "Arrow"</h2>

<p id="uno"></p>
<p id="dos"></p>
```

**Funciones "Arrow"**

2

-2

```
let miF1= function(x, y) {
   return x - y;
}

let miF2= (x, y) => x - y;

document.getElementById("uno").innerHTML= miF1(5, 3);
document.getElementById("dos").innerHTML= miF2(3, 5);
```

# ES6. Clases.

Classes → Nueva forma de definir el constructor de la clase

```
class miClase {
   constructor(uno, dos){
     this.uno= uno;
     this.dos= dos;
   }
}

c= new miClase('v1', 'v2')
```

```
Antes:

function miClase(uno, dos){
          this.uno = uno;
          this.dos = dos;
}
const c1 = new miClase('v1', 'v2');

const c2 = {uno:"v1", dos:"v2"};
```

El acceso sigue haciéndose directamente, mediante el operador "."

```
c.uno
```

**Nota** → *Getters y setters: Ojo los atributos no son privados son todos son públicos, por defecto.*

# ES6. Clases.

Classes → Nueva forma de definir el constructor de la clase

Es posible añadir métodos a la clase, accesibles mediante el operador “.”

Hay dos formas:

```
class miClase {
    constructor(uno, dos){
            this.uno= uno;
            this.dos= dos;
    }
    producto(){
            return this.uno + this.dos;
    }
}

c.producto();
```

# ES6. Clases.

Classes → Nueva forma de definir el constructor de la clase

Es posible añadir métodos a la clase, accesibles mediante el operador “.”

Hay dos formas:

```
class miClase {
    constructor(uno, dos){
            this.uno= uno;
            this.dos= dos;
    }
    get producto(){
            return this.uno + this.dos;
    }
}

c.producto;
```

# ES6. Clases.

Classes → Nueva forma de definir el constructor de la clase

La herencia es posible mediante “extends” e incluyendo “super()” en el constructor:

```
class miOtraClase extends miClase {
    constructor(uno, dos, tres){
            super(uno, dos);
            this.tres= tres;
  }
  ….
}

const ObjOtro= new miOtraClase(v1, v2, v3);
```

# ES6. Mapas y Conjuntos

Map → Tipo de dato creado por pares (clave, valor). Se permite el uso de objetos como claves en un mapa. El acceso es por clave.

| Método | |
|---|---|
| new Map() | Constructor |
| .set() | Fija un valor para un elemento del mapa |
| .get() | Retorna un valor para un elemento del mapa |
| .clear() | Borra todos los elementos del mapa |
| .keys() | Retorna un objeto iterador con todas las claves del mapa |
| .values() | Retorna un objeto iterador con todas los valores del mapa |
| ….. | …… |

```
miMapa= new Map([“cl1”, v1], [“cl2”, v2], …);

miMapa.set(e1, 100);

valor= miMapa.get(e1);
```

# ES6. Mapas y Conjuntos

Set → Se permite la creación de conjuntos (colecciones de datos con valores únicos dentro del conjunto).

| Método | |
|---|---|
| new Set() | Constructor |
| .add() | Añade un valor al conjunto |
| .delete() | Elimina un elemento del conjunto |
| .clear() | Borra todos los elementos del conjunto |
| .values() | Retorna un objeto iterador con todas los valores del conjunto |
| ….. | …….. |

```
miSet= new Set([v1, v2, …])

miSet.add('e1')
```

# ES6. Símbolos.

Tipo Symbol → Nuevo tipo de dato con valor único que permanece oculto

```
class miClase {
   ……
}

let id= Symbol('id')
miClase[id]= 11111;
```

→ `miClase[id]` es accesible, pero `miClase.id` no

# JavaScript ES6

Strings → Nuevos métodos:

- *includes()* → Retorna true si el *string* contiene una cadena dada
- *startsWith()* → Retorna true si el *string* comienza con una cadena dada
- *endWith()* → Retorna true si el *string* finaliza con una cadena dada

# JavaScript ES6

Arrays→ Nuevos métodos:

- *from()* → retorna un array desde un objeto iterable
- *keys()* → retorna un iterador con los índices del array
- *find()* → retorna el primer elemento del array que verifica una función de testeo
- *findIndex()* → retorna el índice del array que verifica una función de testeo

# JavaScript ES6

Otras→ Nuevas funcionalidades para:

- *Objeto Math* → nuevos métodos
- *Objeto Number* → nuevas propiedades y métodos
- *Métodos globales* → dos nuevos métodos (*isFinite(), isNaN()*)

# JavaScript ES6

Modulos→ Ficheros que incluyen colecciones de funciones y/o clases, con un propósito común.

Suelen agruparse en estructuras de carpetas separadas, sin que necesiten un nombre en especial

```
index.html
main.js
modules/
   miModulo_1.js
   miModulo_2.js
```

En los últimos años se ha planteado el uso de la extensión ".mjs" para dichos archivos, pero esto no está todavía generalizado.

# JavaScript ES6

Modulos→ Formas de acceso a los módulos

- Importar nombres exportados desde JavaScript. No se pueden cambiar los nombres al ser importados.

```
import {miF, nombre} from "./miJS.js"
```

Donde en miJS.js:

```
export const nombre= "Pepe"
export function miF()
```

***Nota 1.-*** *existe la posibilidad de cambio de nombre mediante "as" import {miF as mF}*

- Importar nombres exportados desde JavaScript por defecto. Sólo se permite importar uno, pero se puede cambiar el nombre importado.

```
import miFNueva from "./miJS.js"
```

Donde en miJS.js:

```
export default function miF()
```

***Nota 2.-*** *existe la posibilidad de tener un default y varios no default en el mismo módulo*

# JavaScript ES6

## Modulos→ Formas de acceso a los módulos

Para invocar módulos en html hay que incluir el atributo type="module" en la etiqueta <script>

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada: <b id="uno"></b></p>
```

**Funciones JavaScript**

Resultado de la llamada: 7

```
<script type="module">
    import {suma} from "./miModulo.js";
    let x = suma(4, 3);
    document.getElementById("uno").innerHTML = x;
</script>
```

```
export function suma(a, b) {
    return a+b;
}

function resta(a, b) {
    return a-b;
}
```

*miModulo.js*

***Nota 1.-*** *para poder invocar módulos, hay que hacerlo a través de servidores.*

# JavaScript ES6

## Modulos→ Formas de acceso a los módulos

Para invocar módulos en html hay que incluir el atributo type="module" en la etiqueta <script>

```
<h3>Funciones JavaScript</h3>
<p>Resultado de la llamada: <b id="uno"></b></p>

<script type="module" src="miFnc.js"></script>
```

**Funciones JavaScript**

Resultado de la llamada: 7

```
import {suma} from "./miModulo.js";

let x = suma(4, 3);
document.getElementById("uno").innerHTML = x;
```

*miFnc.js*

```
export function suma(a, b) {
    return a+b;
}

function resta(a, b) {
    return a-b;
}
```

*miModulo.js*

***Nota 1.-*** *para poder invocar módulos, hay que hacerlo a través de servidores.*

# Desarrollo de Aplicaciones Web

*JavaScript ES6*

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

*Pablo García Tahoces*